

XML Publisher - Developing reports printed on Pre-Printed Stationary

Abstract

I was inspired to come up with this because I feel that lots of people want this solution and they are struggling to implement this. In this whitepaper, I am going to explain the approach that I used for developing pre-printed stationary reports by XML publisher's RTF method. This approach I used for 2 customers and it works absolutely fine. They are very happy. Example Pre-Printed stationary reports which I worked on are 'AP Check Printing(Bank)', 'AR invoice Printing', 'PO purchase Order Printing', etc.

Background

I have a customer who wants to convert their 40+ Oracle Apps reports to 'XML publisher' Reports. These reports fall in categories of AR invoice report, check printing report and PO print report which they are printing on pre-printed (pre-defined) stationary.

Just a little background of pre-printed stationary report's layout where there is a fix Header section and fix Detail Section Table. Detail section table should always have fixed table height, in the sense it should always have fix number of rows in the table no matter how many rows returned by actual report run. For example Invoice stationary has fix 30 rows in line detail table, and Actual report run is returning only 5 rows then rest 25 blank rows should be generated programmatically.

So far there are solutions available that are talking about fixing table height for the 1st pages onwards, not for the first page itself, i.e. where actual report run is returning lines which are less than lines_fixed_per_page. For example, Invoice run is returning only 5 rows where as pre-printed stationary has fixed 30 lines per page. I struggled a lot to get this solution and now am sharing it in this whitepaper.

So far it has not been discovered because of the limitation of for-loop in XSL-FO (XML Technology). By limitation I mean, we cannot write loop like 'for (i=10;i<15;i++)' in XML. In XML 'for' loop will always iterate till it gets data, if we want to go beyond that then we cannot. We can write for loop based on data value.

To overcome this problem, I have used Sub-template concept, which I am calling recursively.

Detail Solution

I am giving this solution for Standard Check Printing Report. Tree structure of data (Sample XML data is as follows):

```
<LIST_G_CHECKS>
  <G_CHECKS> -- Top Most root -- Header
    <C_CHECK_NUMBER>21897</C_CHECK_NUMBER>
    <C_VENDOR_NUMBER>2205</C_VENDOR_NUMBER>
    <LIST_G_INVOICES>
      <G_INVOICES> -- Inner loop - Line Section
        <C_PAYMENT_NUMBER>1</C_PAYMENT_NUMBER>
        <C_INVOICE_NUMBER>ERS-20-SEP-06-243</C_INVOICE_NUMBER>
      </G_INVOICES>
      <G_INVOICES> -- Inner loop - Line Section
        <C_PAYMENT_NUMBER>2</C_PAYMENT_NUMBER>
        <C_INVOICE_NUMBER>ERS-20-SEP-06-244</C_INVOICE_NUMBER>
      </G_INVOICES>
    </LIST_G_INVOICES>
  </G_CHECKS>
</LIST_G_CHECKS>
```

Below is the step-step guide that I follow.

1) Open the Outermost for loop -- G_CHECKS

```
<?for-each@section:G_CHECKS?>
```

2) Declare Global Variable called 'no_of_lines_per_page' -- In this case I have fixed 40 lines per page.

```
<xsl:variable name="no_of_lines_per_page" select="number(40)"/>
```

3) Declare incontext variable for inner group (G_INVOICES), variable is called 'inner_group'

```
<xsl:variable xdofo:ctx="incontext" name="inner_group" select="//G_INVOICES"/>
```

4) Open the Inner Loop

```
<?for-each:$inner_group?>
```

5) Before putting any elements with the help of current record pointer 'position ()', I am checking if the current position is modulizing with the no_of_lines_per_page equals zero or

not. If it reaches the first record after modulizing then I will create local variable 'first_rec' and initialize it with '0'.

```
<?if:(position()-1) mod $no_of_lines_per_page=0?><xsl:variable name="first_rec"
xdofo:ctx="incontext" select="position()"/>
```

6) If above condition holds true then we will iterate the inner loop.

```
<?for-each:$inner_group?>
```

7) I will check with the help of current record pointer 'position()' that the current record position is either greater than 'first_rec' i.e. the first record or less the 'no_of_lines_per_page' value set up earlier. If it is then show the record otherwise it will not go in loop.

```
<?if:position()>=$first_rec and position()<$first_rec+$no_of_lines_per_page?>
```

8) Here I am closing the inner for loop and if condition

```
<?end if?><?end for-each?>
```

9) Here I am checking if no_of_lines of invoice is modulizing with the no_of_lines_per_page equals to zero or not, and the same time I am checking if \$first_rec+\$no_of_lines_per_page is greater than no_of_lines of invoice or not. This is an important step for filling the blank rows.

```
<?if:not(count($inner_group) mod $no_of_lines_per_page=0) and
($first_rec+$no_of_lines_per_page>count($inner_group))?>
```

10) Now I am calling sub-template recursively for filling the blank rows. While calling this template I am passing one parameter, which is having value of no_of_rows to fill.

Sub-template will have just one row table.

```
<xsl:call-template xdofo:ctx="inline" name="countdown"><xsl:with-param name="countdown"
select="$no_of_lines_per_page - (count($inner_group) mod
$no_of_lines_per_page)"/></xsl:call-template>
```

11) Sub-template declaration

```
<xsl:template name="countdown">
<xsl:param name="countdown"/><xsl:if test="$countdown"><xsl:call-template
name="countdown"><xsl:with-param name="countdown" select="$countdown - 1"/>
</xsl:call-template></xsl:if>
</xsl:template>
```

12) I have created page break after the fixed number of rows have been displayed.

```
<xsl:if xdofo:ctx="inblock"
test="$first_rec+$no_of_lines_per_page<=count($inner_group)">
  <xsl:attribute name="break-before">page</xsl:attribute>
</xsl:if>
```

13) Finally closing outer if and inner for loop and outer for loop.

```
<?end if?><?end for-each?><?end for-each?>
```

If you want more information please contact me at Darshan.bhavsar@gmail.com